Biconnected Components

Jen-Schmidt, Slota-Madduri Algorithms

Vansh Goyal Manik Bansal

May 5, 2026

General Description

- **Slota-Madduri Algorithm:**
  - A parallel BFS approach to find Articulation Points in large graphs.
  - Avoids strict, inherently sequential DFS.
  - Uses BFS and local exclusion traversals to find nodes whose removal disconnects their subgraph.
- **Jen-Schmidt Algorithm:**
  - Computes Chain Decompositions.
  - Strongly sequential, DFS-based framework.
  - Processes non-tree (back) edges to cover tree edges with paths.

Logic Description: Slota-Madduri

- Identifies root components using an initial BFS (assigns parent P and level L).
- Tests if the root is an articulation point by checking if a secondary BFS fails to reach all its children.
- For other nodes, it runs an *exclusion BFS*:
  - Starts from one child w of a candidate vertex v.
  - Traverses without passing through v.
  - If it cannot reach a node at a level $\leq L[v]$, then v is an articulation point.
- **Time Complexity:** Worst-case $O(V \cdot (V + E))$ due to repeated BFS from chokepoints, though faster practically via parallelization.
- **Space Complexity:** $O(V + E)$ total, dominated by graph storage (V, E) and per-thread queues (V).

Logic Description: Jen-Schmidt

- **DFS Traversal:** Builds a spanning forest, keeping track of `parent` and `depth`.
- Classifies non-tree edges as *back edges*.
- **Chain Construction:**
  - Sorts back edges by the highest (shallowest) ancestor reached.
  - Traces upwards from the lower endpoints of back edges using the `parent` pointers.
  - Claims tree edges sequentially until it hits an already claimed edge or the ancestor.
- **Time Complexity:** Strictly $O(V + E)$ due to a single-pass DFS and linear back-edge path tracing.
- **Space Complexity:** $O(V + E)$ total, for graph and back-edge storage, plus $O(V)$ for DFS depth spanning.

Experimental Setup & Dataset Selection

We benchmarked these implementations across various network topologies to expose their time complexity differences.

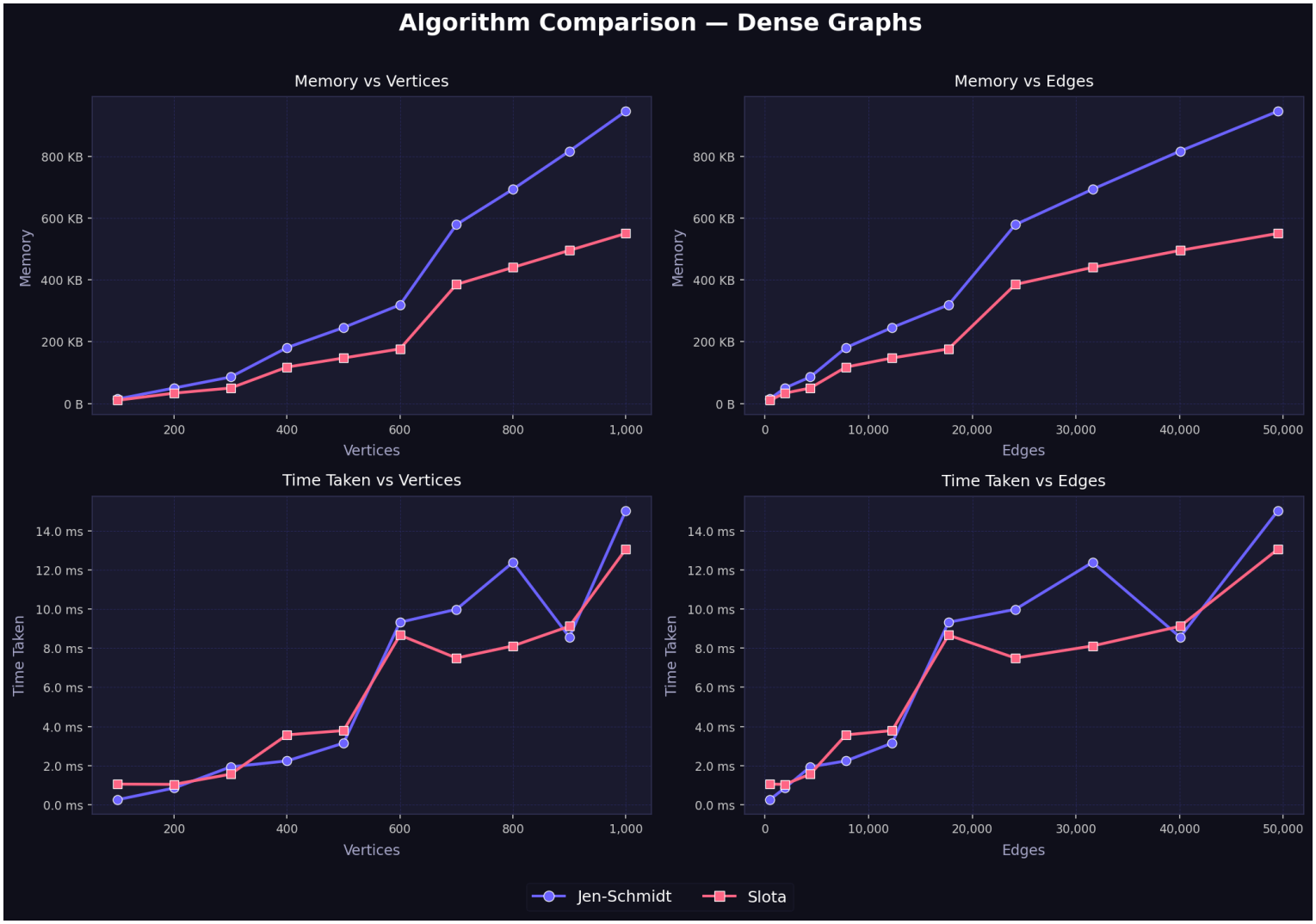
Datasets:

1. **Small:** Quick baselines.
2. **Large:** High V and E.
3. **Dense:** High structural redundancy; almost complete subgraphs.
4. **Sparse:** Average degree is small; extended path lengths.
5. **Highly Connected:** High degree uniform nodes.
6. **Tree-Like:** Adversarial deep paths, very few back edges.
7. **Real-World:** Empirical data (e.g., social networks, github, power grids).

Dense Graphs

- **Expected Behavior:** Slota better in Time and Memory. Dense graphs generate many back edges, making Jen-Schmidt DFS heavy and less parallelizable, creating many chains. Slota handles large frontier expansions in parallel.
- **Observed Behavior:** Matches expectations. Slota utilizes parallel processing efficiently reaching rapid success states. Jen-Schmidt uses more time and moderately higher memory storing massive back edge counts.

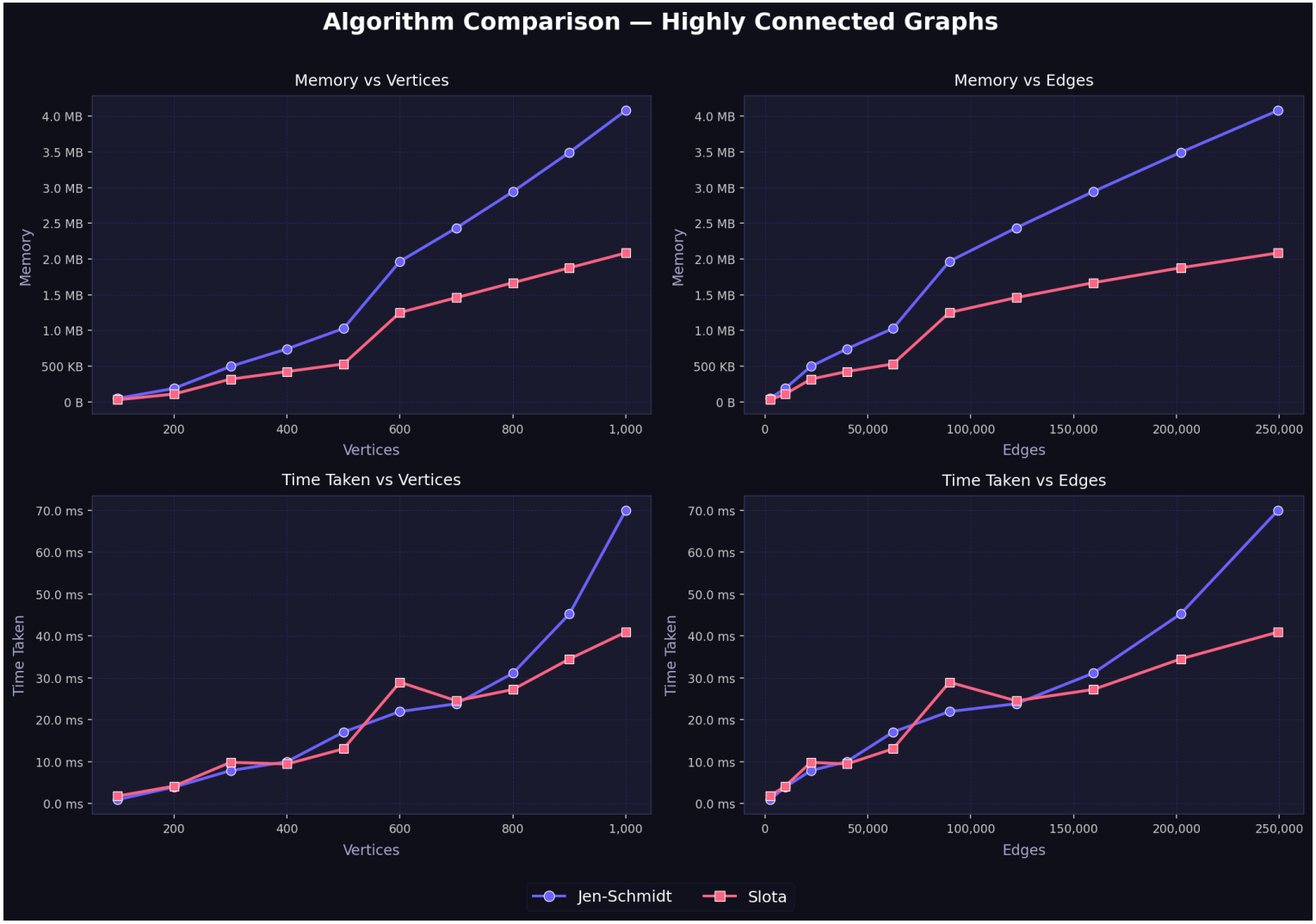
Dense Graphs



Highly Connected Graphs

- **Expected Behavior:** Slota noticeably better. High cyclic structures boost back edges and chain creation, increasing Jen-Schmidt DFS dependency. Slota scales better under high parallelism. Memory is slightly higher for Jen-Schmidt.
- **Observed Behavior:** Matches expectations. Slota's exclusion searches succeed quickly, proving highly scalable. Jen-Schmidt incurs moderately higher memory storing numerous chains compared to Slota's compact parallel structures.

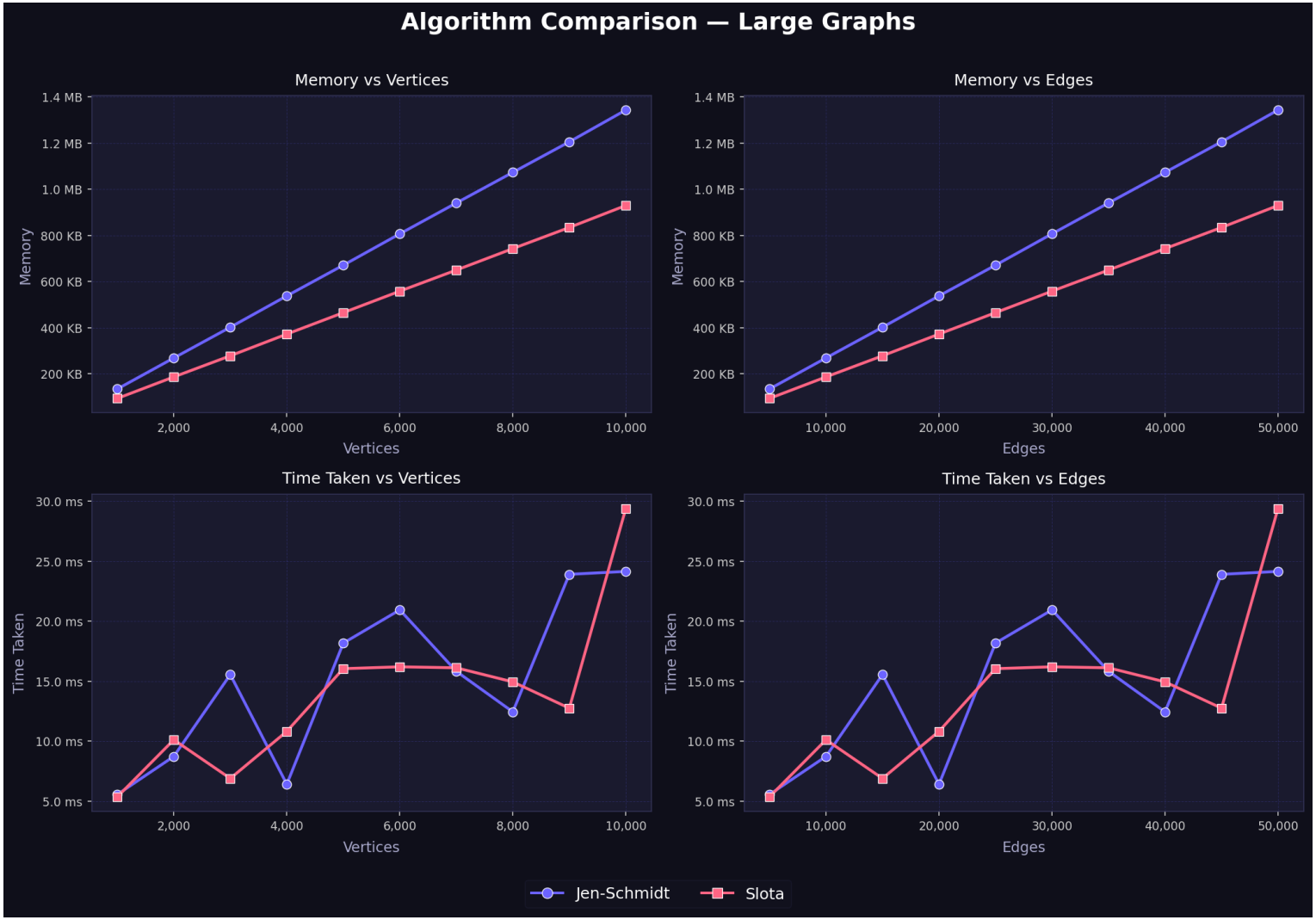
Highly Connected Graphs



Large Graphs

- **Expected Behavior:** Slota-Madduri should be significantly better at scale due to parallel BCC design. Jen-Schmidt is expected to be limited by sequential DFS bottlenecks and cache inefficiencies. Memory scalability is comparable.
- **Observed Behavior:** Slota utilizes the cores and scales well on broad hubs, while Jen-Schmidt scales optimally strictly dependent on size, often staying broadly competitive depending on the underlying diameter.

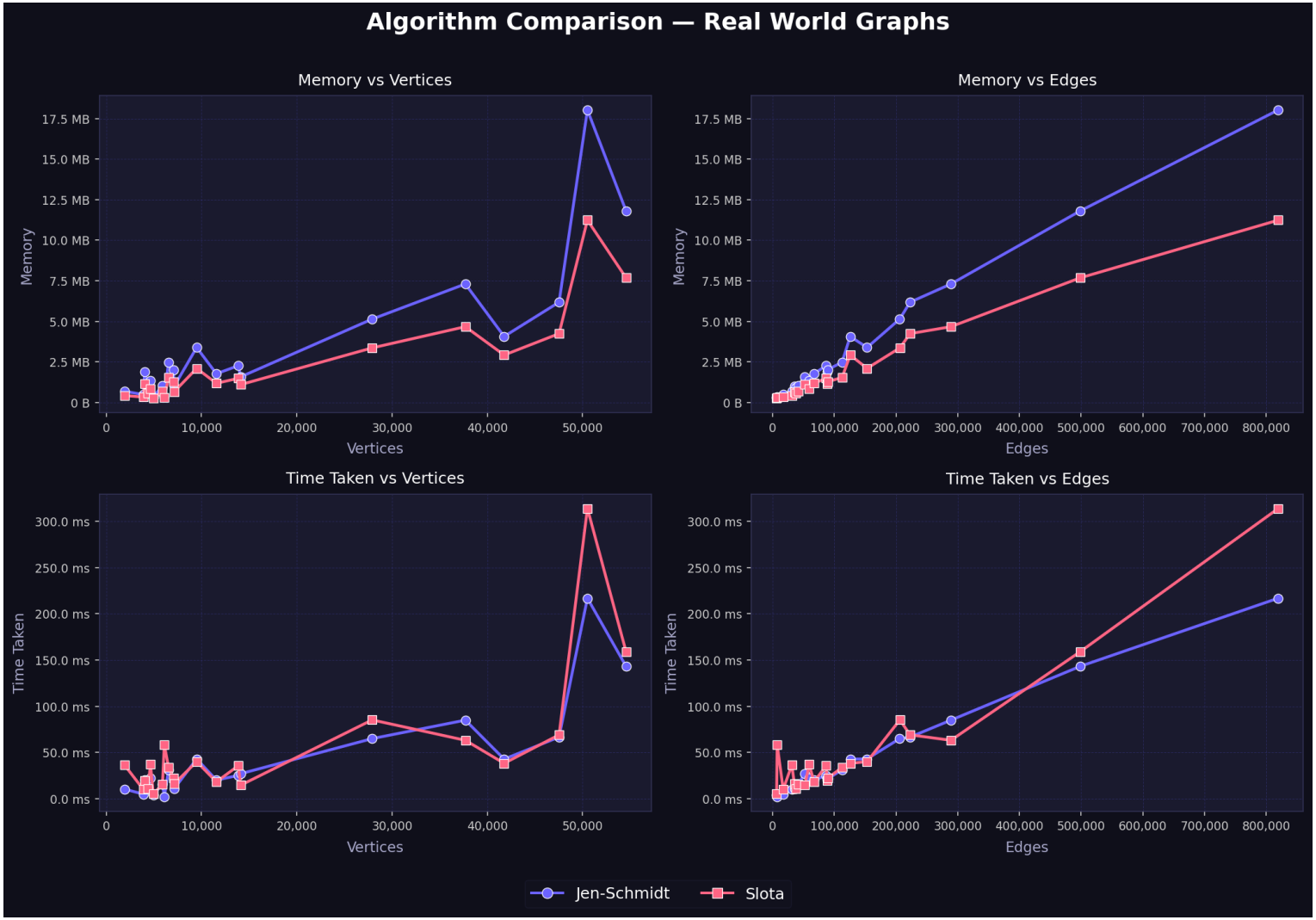
Large Graphs



Real-World Graphs

- **Expected Behavior:** Slota generally better. High-degree hubs create many back edges, making Jen-Schmidt chains expensive. Slight memory edge to Slota on massive scale-free networks.
- **Observed Behavior:** Highlights practical nuances. Slota benefits exceptionally from parallel edge processing on dense social hubs. Jen-Schmidt remains highly competitive mapping the sparse, power-law regions.

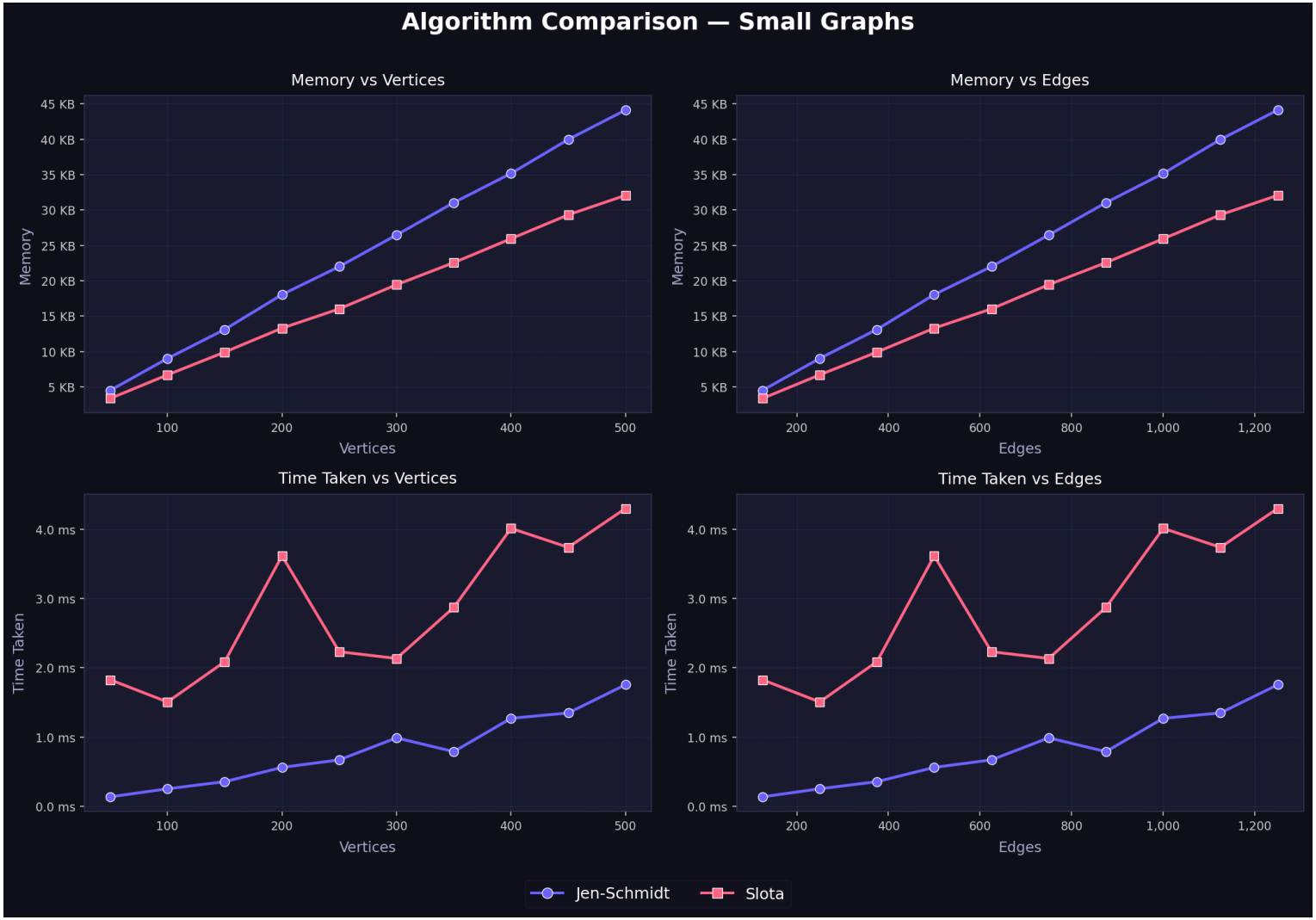
Real-World Graphs



Small Graphs

- **Expected Behavior:** Executions are nearly identical, but Jen-Schmidt should appear faster as Slota-Madduri incurs parallel setup overhead. Memory is minimal for both.
- **Observed Behavior:** Matches expectations. Both complete in $< 0.01s$, but Jen-Schmidt is faster as it avoids thread-spawning and synchronization overheads.

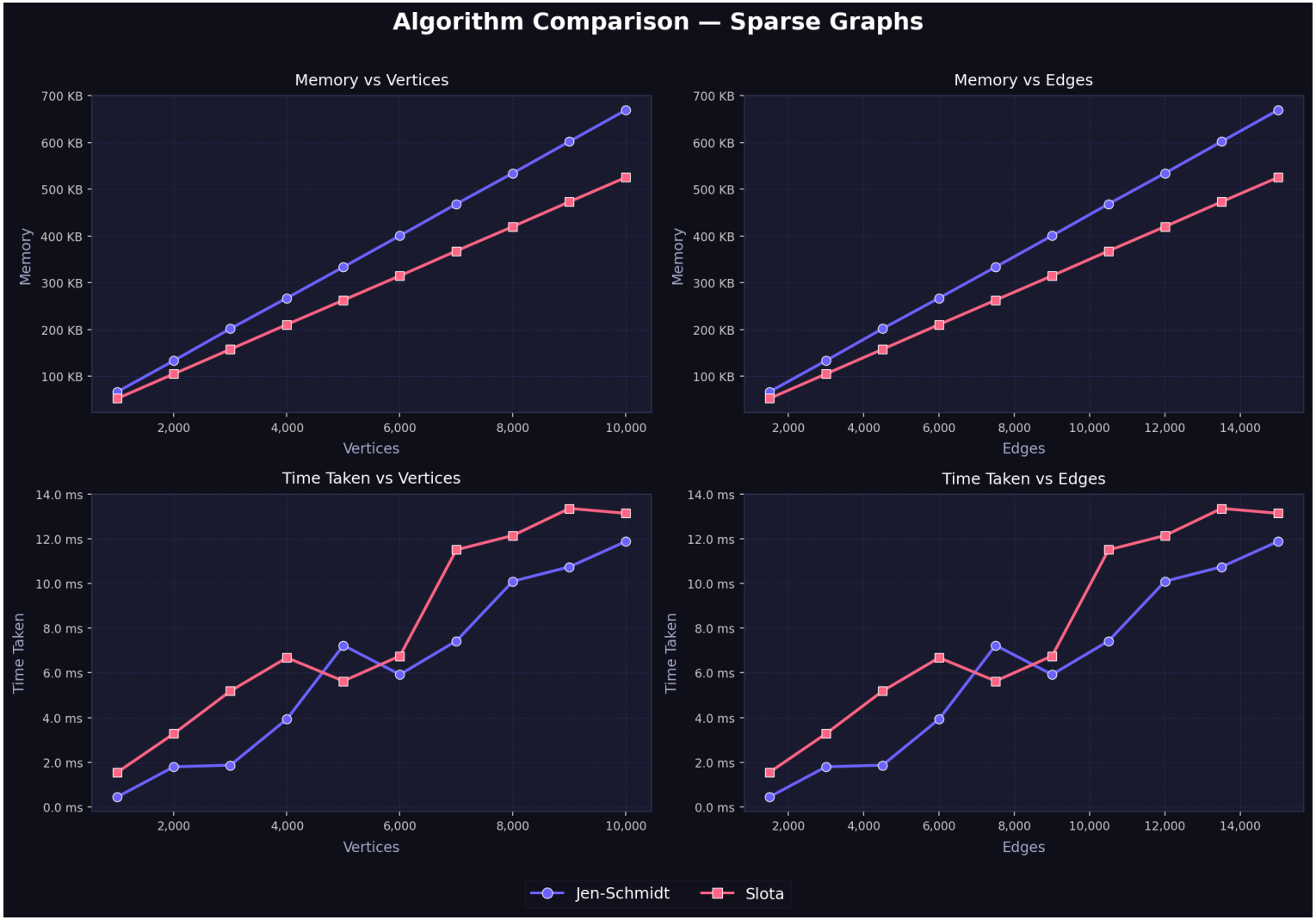
Small Graphs



Sparse Graphs

- **Expected Behavior:** Jen-Schmidt highly competitive or better in Time. Minimal back edges keep DFS lightweight, limiting Slota's parallel advantages. Memory usage is similar.
- **Observed Behavior:** Matches expectations. Jen-Schmidt chain decomposition works very efficiently in a single pass. Slota occasionally suffers if paths to low depths are artificially restricted.

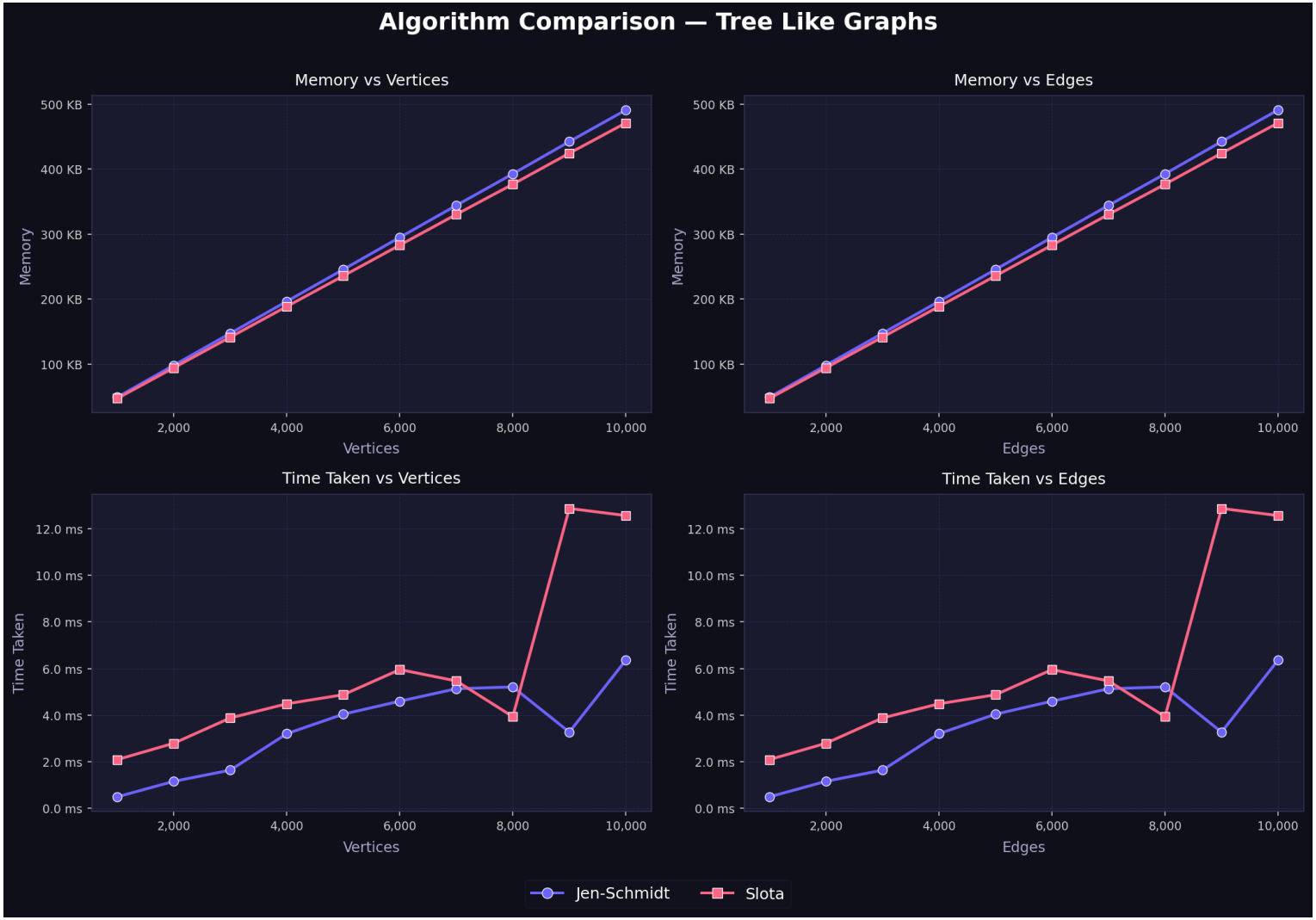
Sparse Graphs



Tree-Like Graphs

- **Expected Behavior:** Heavily favors Jen-Schmidt. Chain decomposition becomes trivial with almost no chains/cycles to track. Slota still pays framework overhead. Both utilize low memory.
- **Observed Behavior:** Amplified expectations. Jen-Schmidt execution drops to near $0s$ due to skipped back-edge processing loops. Slota hits worst-case $O(V \cdot E)$ bottlenecks traversing deep sub-trees continuously.

Tree-Like Graphs



Thank You!